

PROCESS SYNCHRONISATION

①

```

define N 100
int Buffer[N];
int count=0; <no. of data items in buffer>
void Producer(void){
    int item, in=0;
    while(1){
        item = produce_item();
        while(count==N);
        Buffer[in] = item;
        in = (in+1) % N;
        count = count+1;
    }
}
void Consume(void){
    int item, out=0;
    while(1){
        while(count==0);
        item = Buffer[out];
        out = (out+1) % N;
        process_item(item);
    }
}

```

(1) LOCK variable → glw solⁿ
 → busy waiting → Multi-Process solⁿ

```

int Lock=0; void Process(int i) {

```

```

a) <NCS>
b) while (lock!=0);
   lock=1;
c) <CS>
d) lock=0

```

ME violated

Prog
P1 P2
Box

(2) STRICT ALLOCATION → BW → 2 process solⁿ

```

int turn = rand(0,1)
void Process(int i)
while(1){

```

```

a) NCS
b) while (turn!=i);
c) <CS>
d) turn=i;
}

```

```

void Process(int i)
while(1){

```

```

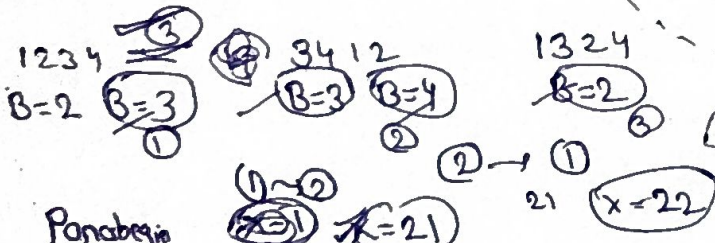
a) NCS
b) while (turn!=i);
c) <CS>
d) turn=i;
}

```

ME ✓ Progress BW

1. C = B - 1
 2. B = 2 * C

3. D = 2 * B
 4. B = D - 1



~~Process P1 and P2 are not equal to BW~~
 BW → P1 ki jake kisi ki process CS me jake wali hai

violates Progress
 But not turn-wise
 21 2 22

Panabegin

```

begin S1; v(a); v(b); end
begin P(a); S2; S4; v(c); v(d); end
begin P(b); S3; v(e); end
begin P(c); S5; v(f); end
begin P(d); P(e); S6; v(g)

```

end begin: P(f); P(g); S7;

```

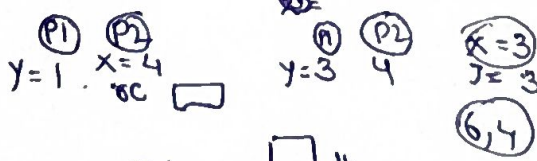
void Producer(void)
{
    int item, in=0;
    while(1){
        item = produce_item();
        Down(Empty);
        Down(mutex);
        Buf[in] = item;
        in = (in+1) % N;
        Up(mutex);
        Up(full);
    }
}

```

```

Down(Full)
Down(mutex);
item = Buf[out];
out = (out+1) % N;
Up(mutex);
Up(empty);
}

```



rw
 int rc=0;
 BSEM mutex = 1; <used by rc>
 BSEM db = 1; <Row's Access to db>

```

writer(void){
    Down(db)
    <WRITE>
    Up(db)
}

```

```

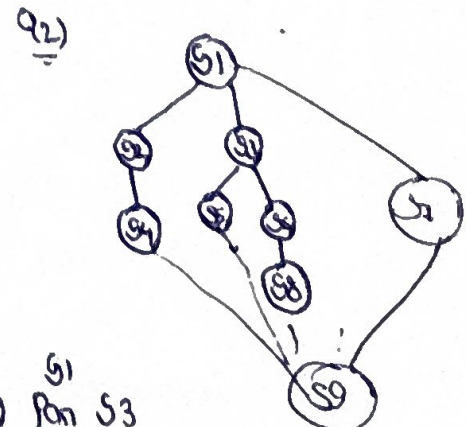
reader(void){
    Down(mutex)
    rc = rc+1;
    if (rc==1) down(db);
    Up(mutex);
    <READ>
    Down(mutex)
    rc = rc-1;
    if (rc==0) up(db);
    Up(mutex);
}

```

```

S1
Panbe S3;
begin S2; S4;
Pan S5; S6;
end
Pan S7;

```



S1
 Q Pan S3
 be S2; S4; S5 end
 Panend

E N
 full → 0

3) PETERSON'S ALGO → Busy waiting

FIFO → 2 process soln
→ LOCK var + STRICT ALT

```
#define N 2
#define TRUE 1
#define FALSE 0
int flag[N] = {FALSE};
int turn;

void process(int i) {
    int j = NOT(i);
    while(1) {
        a) NCS
        b) flag[i] = TRUE;
        c) turn = i;
        d) while (FLAG[j] == TRUE & turn == j);
        e) <CS>
        f) flag[i] = FALSE;
    }
}
```

i) M/E ✓
ii) Progress ✓ [if j flag FALSE then direct] (CS)
iii) BW ✓

II SYNCHRONIZATION H/W → Busy waiting

→ OS as special instn → H/W category
→ Multiprocess soln → lock based soln

G' TSL (Test and Set lock)

return current value of lock & set value of lock to TRUE

```
Bool TSL (Bool *target) {
    Bool tv;
    *tv = *target;
    *target = TRUE;
    return(tv);
}
```

ATOMIC EXECUTION (hm)

```
Bool lock = F;
void process(int i) {
    while(1) {
        a) NCS
        b) while (TSL(&lock) == 1);
        c) <CS>
        d) lock = F;
    }
}
```

M/E ✓ Progress BW (AL)

void Dekkers - Algo (int i) {

int j = !i;

while(1)

a) NCS

b) flag[i] = TRUE

c) while (!flag[j] == TRUE) {
if (turn == j) {

flag[i] = false;

while (turn == j);

flag[i] = TRUE;

}

d) <CS>

e) turn = j;

f) flag[i] = false;

}

i) M/E ✓
ii) Progress ✓
iii) BW ✓

b) SWAP Instn

```
SWAP (Bool *a, Bool *b)
{
    Bool t;
    t = *a;
    *a = *b;
    *b = t;
}
```

Bool lock = F;

void process(int i) {

Bool key = T;

while(1)

a) NCS

b) while (key == T)

SWAP(&lock, &key);

c) <CS>

d) lock = F;

}

M/E

Progress BW (with lock)

lock → T
key → F
F SWAP T
[lock] [key]
anyone present in CS?
Intended?

Blocking Mechanism / Non-Busy waiting

1) Sleep() & wakeup() → multipro. soln
→ OS primitive (km) → Sleep() → process block
till some other ps. wake
it up.

define N 100

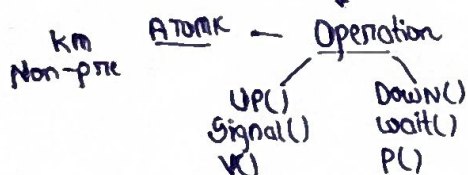
int Buffer[N];

int count = 0;

```
void Producer(void) {
    int item, in = 0;
    while(1) {
        a) item = Produce-item();
        b) if (count == N) sleep();
        c) Buffer[in] = item;
        d) in = (in + 1) % N;
        e) count = count + 1;
        f) if (count == 1) wakeup(consumer);
    }
}
```

SEMAPHORES → OS resource (S/W)

ADT (Abstract Datatype)



→ ADT that takes int values

```

    (BSEM) BINARY (mutex) (0, 1)
           |
           +--- COUNTING (General) (-∞, +∞)
  
```

COUNTING SEMAPHORE

```

Type def struct {
    int values;
    QueueType L;
} CSEM;
  
```

list of PCB's of those process
that gets blocked, while performing
"Down" ops unsuccessfully

```

um CSEM S;
S.value = 1;
Down(S);
  
```

3

Empty

```

T1: C → if (count == 0) ✓ Proc.
P1 → a, ..., e, f → wakeup
C → Sleep()
P2 → a, ..., e, f (false)
P100 → a, e, ...
P101 → a, b
  
```

Deadlock

```

void Consumer(void) {
    int itemc, out = 0;
    while(1) {
        a) if (count == 0) sleep();
        b) itemc = Buffer[out]; out = (out + 1) % N;
        c) count = count - 1;
        d) if (count == N - 1) wakeup(producer);
        e) process-item(itemc);
    }
}
  
```

km DOWN (CSEM S) {

```

1. S.value = S.value - 1;
2. if (S.value < 0) //unsuccessful
   { put the process's PCB in
     SL(Q) & block the process (sleep) }
   else
     return;
}
  
```

CSEM S; S = 5 S = 4 3 2 1 0 -1 -2
5 succ. [|||||] 2 unav.

NOTE 1) +ve value of semaphore indicates that those
many down ops can be carried out successfully
2) -ve → indicates no. of blocked process

UP (CSEM S) {

```

1. S.value = S.value + 1;
2. if (S.value < 0)
   { select a process from SL(Q)
     & wakeup();
   }
}
  
```

CSEM S = 8
10P, 1V, 15P, 2V, 6P, 3V

Cases of Binary Semaphore [process only gets blocked performing Down opⁿ]

- 1) BSEM S;
S=1;
P(S);
→ S=0
Status = Success
- 2) BSEM S;
S=0;
P(S);
→ S=0
Status = Unsuccess
- 3) BSEM S;
S=0;
V(S);
S { 0 → if SQ is not empty
1 → if sleeping Q is empty
Status = Success
- 4) BSEM S;
S=1; (is free & Q empty)
V(S);
→ S=1;
Status = Success

* 5) BSEM;
S=0; <Initial>
V(S);
S=1
Status = Success

* first opⁿ <'Q' is empty>